Company Policies & FAQs

NovaTech Systems

Company Policies

Warranty Policy

- All laptops come with a **1-year limited hardware warranty** from the date of purchase.
- Warranty covers:
    - Manufacturing defects
    - Hardware malfunctions
    - Faulty components
- Warranty does **not** cover:
    - Accidental damage
    - Liquid spills
    - Unauthorized repairs
    - Cosmetic issues
- **Extended warranty** and **accidental damage protection plans** are available at an additional cost.

Return & Replacement Policy

Customers can request a **return or replacement within 7 days of delivery** if the product:

- Is **dead on arrival**
- Has a **major defect**
- Is delivered in a **damaged condition**

For **minor issues**, repairs will be provided under warranty instead of a full replacement.

Refund Policy

Refunds are processed only if:

- The product is returned in **original packaging and condition**
- The return request is raised **within 7 days of delivery**
- The product is:
    - Defective

    - Damaged during shipping
    - Delivered incorrectly
- Refunds are credited to the **original payment method** within **7–10 business days** after inspection.
- Refunds are **not applicable** for preference-based returns (e.g., color choice or performance dissatisfaction).

Repair & Service Policy

- All repairs are handled by **authorized service centers**.
- Customers must present:
    - Original invoice
    - Warranty card
- **Out-of-warranty repairs** will be charged as per standard rates.

Shipping & Delivery Policy

- Standard delivery time: **3–7 business days** within serviceable regions.
- **Free shipping** on orders above **₹50,000**.
- Tracking details are shared via **email/SMS** once the order is dispatched.

Customer Support Policy

Support is available through:

- **Toll-free helpline:** 9 AM – 9 PM IST
- **Email support:** 24/7 ticket system
- **Live chat:** Official website

On-site support is available for **select premium models**.

Frequently Asked Questions (FAQ)

Product & Warranty

What does the standard warranty include?
The warranty covers manufacturing defects and hardware failures. It does not include accidental damage, physical wear and tear, or software issues caused by third-party applications.

How do I register my laptop for warranty?
You can register your laptop on our official website using the product serial number and purchase invoice. Registration is recommended within **30 days of purchase**.

Can I upgrade my laptop's RAM or storage?
Yes, most models allow upgrades. However, upgrades must be done at an **authorized service center** to avoid voiding the warranty.

Technical Issues

What should I do if my laptop won't turn on?

- Check if the power adapter is properly connected
- Try charging using a different power outlet
- If the issue persists, contact customer support or visit an authorized service center

Do you provide international warranty?
Yes, **premium series laptops** include international warranty coverage in select countries. Please check the official website for eligible regions.

How often should I service my laptop?
A basic health check is recommended **every 12 months** at an authorized service center.

Orders & Delivery

How can I track my order?
Once shipped, you will receive a tracking ID via email/SMS. Use it on our logistics partner's website to track your order.

Where can I buy original accessories?
Original accessories (chargers, batteries, docking stations) are available through:

- The official website
- Authorized resellers

Refunds & Returns

Can I return my laptop if I don’t like it?
No. Returns are accepted only for:

- Manufacturing defects
- Damage during delivery
- Incorrect product delivery

Personal preferences (design, performance, or color) are not valid reasons.

How long does it take to get a refund?
Refunds are processed within **7–10 business days** after inspection.

Will I get a full refund if my laptop is returned?
Yes, provided the laptop is returned:

- In original packaging
- With all accessories
- Meeting return conditions

Deductions may apply if items are missing or damaged.

What if I receive a defective laptop?
If the laptop is dead on arrival or has a major defect, you may request a **full refund or replacement within 7 days of delivery**.

PASTE IN CHATGPT AND GET YOURSELF THE COMPLETE CODE

Prompt to Generate: Naive RAG Pipeline — Lecture 1 Project

Instruction

You are a Python AI developer and educator. Build a complete, beginner-friendly RAG (Retrieval-Augmented Generation) pipeline in a **single Python notebook (.ipynb)** from scratch — no LangChain, no LlamaIndex, no frameworks. Use only direct Python libraries.

The audience is working professionals who know basic Python but have zero knowledge of RAG, embeddings, vector databases, or LLMs. Every step must be explained with comments, print statements showing intermediate outputs, and markdown cells explaining the "why" before each code cell.

Project Name: "Document RAG Assistant"

Project Goal

Build a pipeline that:

1. Loads a PDF and a TXT file
2. Cleans and preprocesses the text
3. Chunks the text using multiple strategies
4. Generates embeddings using OpenAI API
5. Stores embeddings in ChromaDB
6. Retrieves relevant chunks for a user query
7. Sends retrieved context + query to GPT-4o-mini
8. Returns a grounded answer

Step-by-Step Requirements

Step 1: Setup & Installations

- Install: `pypdf2` or `pdfplumber`, `openai`, `chromadb`, `numpy`
- Import all libraries
- Set OpenAI API key (use `os.environ` or direct variable)

Step 2: Document Loading

- Load a **PDF file** using `PyPDF2` or `pdfplumber` — extract text page by page
- Load a **TXT file** using plain Python `open()`
- Structure each loaded document as a Python dictionary:

{"content": "extracted text here...", "metadata": {"source": "filename.pdf", "page": 1}}

- Store all documents in a list called `documents`
- **Print:** Show the first 500 characters of each loaded document and its metadata
- **Print:** Total number of documents loaded

Step 3: Text Preprocessing

- Write a `clean_text()` function that:
  - Strips extra whitespace and newlines
  - Removes special/junk characters
  - Lowercases the text (optional, explain trade-off)
  - Fixes encoding issues
- Apply `clean_text()` to all documents
- Keep metadata attached to each document
- **Print:** Show before vs after cleaning for one document

Step 4: Chunking (Implement 3 Strategies)

Strategy 1: Fixed-size character chunking

- Write a function `fixed_size_chunk(text, chunk_size=500)`
- Split text into chunks of exactly `chunk_size` characters
- **Print:** Number of chunks, first chunk

Strategy 2: Sliding window chunking with overlap

- Write a function `sliding_window_chunk(text, chunk_size=500, overlap=100)`
- Split text with overlapping windows
- **Print:** Number of chunks, show how overlap works between chunk 1 and chunk 2

Strategy 3: Recursive chunking

- Write a function `recursive_chunk(text, chunk_size=500)`
- Logic: First try to split by `\n\n` (paragraphs), then by `.` (sentences), then by character limit
- This is the most important one — explain why it preserves meaning better
- **Print:** Number of chunks, first 3 chunks

Choose recursive chunking as the default for the rest of the project.

- Each chunk should be a dictionary:

{"content": "chunk text...", "metadata": {"source": "filename.pdf", "page": 1, "chunk_id": 0}}

- **Print:** Total chunks created, sample chunk with metadata

Step 5: Embeddings

- Explain what embeddings are (add a markdown cell: "Turning text into numbers that capture meaning")
- Write a function `get_embeddings(texts)` that:
  - Calls `openai.embeddings.create(model="text-embedding-3-small", input=texts)`
  - Returns a list of embedding vectors
- Generate embeddings for ALL chunks (batch if needed)
- Also generate an embedding for a sample query to show it's the same process
- **Print:** Shape/length of one embedding vector
- **Print:** "Each chunk is now a vector of {length} dimensions"

Step 6: Vector Store with ChromaDB

- Explain what a vector database is (markdown cell: "A search engine for meaning, not keywords")
- Create a ChromaDB client (in-memory or persistent)
- Create a collection called `"document_rag"`
- Add all chunks to ChromaDB with:
  - `documents`: chunk text content
  - `embeddings`: the embedding vectors

    - `metadatas`: the metadata dicts
    - `ids`: unique string IDs like `"chunk_0"`, `"chunk_1"`, etc.
- **Print:** "Stored {n} chunks in ChromaDB"
- **Print:** Collection count to verify

Step 7: Retrieval

- Explain cosine similarity in simple terms (markdown cell: "Finding the closest meaning match")
- Write a function `retrieve(query, n_results=3)` that:
    - Generates embedding for the query using the same `get_embeddings()` function
    - Queries ChromaDB with `collection.query(query_embeddings=..., n_results=...)`
    - Returns the retrieved documents, distances/scores, and metadata
- Test with 2-3 sample queries
- **Print:** For each query, show:
    - The query
    - Each retrieved chunk (numbered)
    - The distance/score for each chunk
    - The source metadata for each chunk

Step 8: LLM + Prompt Engineering

- Explain the RAG prompt pattern (markdown cell)
- Write a function `build_prompt(query, retrieved_chunks)` that:
    - Takes the query and list of retrieved chunk texts
    - Builds a prompt string with:
        - **System message:** "You are a helpful assistant. Answer questions ONLY based on the provided context. If the answer is not in the context, say 'I don't have enough information to answer this.'"
        - **User message:** Contains the context (numbered chunks) and the question
    - Prompt format:

Context:
[1] {chunk_1_text}
[2] {chunk_2_text}
[3] {chunk_3_text}

Question: {user_query}

Answer based only on the context above. If the context doesn't contain the answer, say "I don't have enough information to answer this."

- **Print:** Show the full constructed prompt for one query

- Write a function `ask_llm(query, retrieved_chunks)` that:
    - Builds the prompt
    - Calls `openai.chat.completions.create()` with:
        - `model="gpt-4o-mini"`
        - `messages=[{"role": "system", "content": system_prompt}, {"role": "user", "content": user_prompt}]`
        - `temperature=0.2` (low for factual answers)
    - Returns the LLM's answer

Step 9: Complete Pipeline Function

- Write a single function `rag_pipeline(query)` that:
    1. Takes a user question
    2. Retrieves relevant chunks
    3. Builds the prompt
    4. Calls the LLM
    5. Returns the answer
- **Print** the full flow:

📝 Query: {query}
🔍 Retrieved {n} relevant chunks
📄 Sources: {list of sources}
🤘 Answer: {llm_response}

Step 10: Live Demo — Test the Pipeline

- Run the pipeline with 5 different questions:
    - 2 questions that should be answerable from the documents
    - 1 question that tests across both PDF and TXT sources
    - 1 question that is partially answerable
    - 1 question that is NOT in the documents (to test hallucination control)
- For each question, display:
    - The question
    - Retrieved chunks (abbreviated)

    - The LLM's answer
    - Which source documents were used

Step 11: Summary & Recap Cell

- Add a final markdown cell that summarizes:
    - The full pipeline: Load → Clean → Chunk → Embed → Store → Retrieve → Prompt → Answer
    - Libraries used: `PyPDF2`/`pdfplumber`, `openai`, `chromadb`, `numpy`
    - Key decisions made (chunk size, overlap, embedding model, retrieval K, temperature)

Code Style Requirements

- **Every function** must have a docstring explaining what it does
- **Every step** must have a markdown cell BEFORE the code explaining the concept in 2-3 simple sentences
- **Print intermediate outputs** at every step so students can see what's happening
- Use **simple variable names** — no complex abstractions
- Add `# comments` generously inside code
- No classes — keep everything as functions and simple data structures (dicts, lists)
- Use **f-strings** for prompt building
- Handle errors gracefully with try/except where API calls are made

File Requirements

- Use any PDF (can use a sample or placeholder — suggest the user to replace with their own file)
- Use any TXT file (can create a sample about a fictional company or use placeholder)
- If no real files are available, **create sample text content inline** as a fallback so the notebook runs without external files

Do NOT Use

- LangChain
- LlamaIndex
- Any RAG framework
- Any wrapper libraries
- Classes or OOP patterns
- Complex abstractions

Build everything from raw Python + the specified libraries only.